

ITF23019: Parallel and Distributed Programming

Report for Lab 5: When to use Parallel Programming

Name: Emil Berglund

GitHub: EmilB04

Exercise 1: Vector vector mul project

Subtask 1: Use ForkJoin framework to complete parallel implementation in *VectorVectorMulTask.java* and *VectorVectorMulParallel*.

DONE

Subtask 2: When the output of your parallel program implementation and that of the serial program are the same, write code the Main class to compute the speedup of your parallel program. Include the output in your report.

```
Parallel-distributed-programming/lab5 on  main [!?]
> cd /home/emilb_linux/dev/V2026/Parallel-distributed-p
m/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInE
code-server/data/User/workspaceStorage/bbc052fc79e6d3b33
b18db9/bin main.Main
Serial program is running...
Serial program returns 2102 during 310461 (ns)
Parallel program is running ...
Parallel program returns 2102 during 4319190 (ns)
Speedup is 0.07187944961902579

Parallel-distributed-programming/lab5 on  main [!?]
> |
```

Subtask 3: Fix the value of threshold in the *VectorVectorMulTask* class and change the size of the two input vectors to see how the speedup varies. Include several outputs in your report.

N = 200

```
Parallel-distributed-programming/lab5 on  main [!?]
> cd /home/emilb_linux/dev/V2026/Parallel-distribut
m/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetail
code-server/data/User/workspaceStorage/bbc052fc79e6d
b18db9/bin main.Main
Serial program is running...
Serial program returns 4392 during 289580 (ns)
Parallel program is running ...
Parallel program returns 4392 during 5814637 (ns)
Speedup is 0.04980190508883014
```

N = 500

```
Parallel-distributed-programming/lab5 on main [!?]
> cd /home/emilb_linux/dev/V2026/Parallel-distributed-programming/lab5
m/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetails -ea -cp
code-server/data/User/workspaceStorage/bbc052fc79e6d3b18db9/bin
main.Main
Serial program is running...
Serial program returns 9499 during 269187 (ns)
Parallel program is running ...
Parallel program returns 9499 during 7790905 (ns)
Speedup is 0.034551441713125755
```

N = 1500

```
Parallel-distributed-programming/lab5 on main [!?]
> cd /home/emilb_linux/dev/V2026/Parallel-distributed-programming/lab5
m/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetails -ea -cp
code-server/data/User/workspaceStorage/bbc052fc79e6d3b18db9/bin
main.Main
Serial program is running...
Serial program returns 30690 during 320027 (ns)
Parallel program is running ...
Parallel program returns 30690 during 6341447 (ns)
Speedup is 0.05046592678295663
```

Subtask 4: Run your implementation with different values of threshold while keeping the size of the input vectors fixed to see how the speedup varies. Include several outputs in your report.

Threshold = 10

```
Parallel-distributed-programming/lab5 on main [!?]
> /usr/bin/env /usr/lib/jvm/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetails -ea -cp
s -cp /home/emilb_linux/.vscode-server/data/User/workspaceStorage/bbc052fc79e6d3b18db9/bin
/redhat.java/jdt_ws/lab5_e7b18db9/bin main.Main
Serial program is running...
Serial program returns 4144 during 276081 (ns)
Parallel program is running ...
Parallel program returns 4144 during 7048117 (ns)
Speedup is 0.03917088777044989
```

Threshold = 30

```
Parallel-distributed-programming/lab5 on main [!?]
> /usr/bin/env /usr/lib/jvm/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetails -ea -cp
s -cp /home/emilb_linux/.vscode-server/data/User/workspaceStorage/bbc052fc79e6d3b18db9/bin
/redhat.java/jdt_ws/lab5_e7b18db9/bin main.Main
Serial program is running...
Serial program returns 3491 during 376128 (ns)
Parallel program is running ...
Parallel program returns 3491 during 8081408 (ns)
Speedup is 0.04654238469336037
```

Threshold = 80

```
Parallel-distributed-programming/lab5 on main [!?]
> /usr/bin/env /usr/lib/jvm/java-21-openjdk-amd64/bin/java -cp /home/emilb_linux/.vscode-server/data/User/workspaceStorage/bbc052fc79e6d3b3e7b18db9/bin main.Main
Serial program is running...
Serial program returns 3972 during 244888 (ns)
Parallel program is running ...
Parallel program returns 3972 during 2143146 (ns)
Speedup is 0.11426566365520595
```

Subtask 5: Set an extreme value for the vector size and observe what happens when you run your program.

$N = 2\,000\,000$ (2 million)

```
Parallel-distributed-programming/lab5 on main [!?]
> cd /home/emilb_linux/dev/V2026/Parallel-distributed-programming/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInExceptionReport -cp /home/emilb_linux/.vscode-server/data/User/workspaceStorage/bbc052fc79e6d3b3e7b18db9/bin main.Main
Serial program is running...
Serial program returns 40531286 during 3714780 (ns)
Parallel program is running ...
Parallel program returns 40531286 during 71790098 (ns)
Speedup is 0.051745019208637937
```

Exercise 2: Matrix-matrix-mul project

Disclaimer: I found this task to be quite hard, and I therefore got some help from ChatGPT 5.2 for completing the parallel implementation.

Subtask 1: Use ForkJoin framework to complete parallel implementation. (The provided code uses RecursiveAction to implement Task in the ForkJoin framework, but you are free to use RecursiveTask - modify the starter code accordingly.)

DONE

Subtask 2: The compareResults(resultSerial, resultParallel) in the starter code is used to check whether your serial and parallel implementations produce the same output. If they do, your two implementations are correct, and you can proceed to write your code to compute the speedup of your parallel program. Include the output in your report. Otherwise, you need to debug your code to fix your implementation until both serial and parallel versions are correct.

Subtask 4: The run complexity of Exercise 1 is $O(n)$ while that of Exercise 2 is $O(n^3)$. Suppose that your serial and parallel implementations in both two exercises are reasonable, and that you use the same data size (for example, N elements in each vector in Exercise 1 and N rows/columns in each matrix in Exercise 2). Which exercise results in a better speedup? Explain your answer.

Exercise 2 (matrix–matrix) gives better speedup.

- Exercise 1 (vector–vector, $O(n)$) does little work per data element, so thread/startup/communication costs are big compared to the useful work.
- Exercise 2 (matrix–matrix, $O(n^3)$) does a lot of work per data element, so the same overhead is “hidden” by much more computation, letting the parallel version gain more from multiple cores.